Project Genesis: How the Contract Copilot Idea Arose

This report documents the conceptual inception, technical reasoning, prompt parameter configurations, and iterative corrections that shaped the development of the **SecureFlow Contract Copilot** system.

💡 1. The Inception of the Idea

The concept arose from a recurring pain point in corporate legal departments: **the contract review bottleneck**. Legal teams are regularly swamped with boilerplate Mutual NDAs and Customer Order Forms. While 80% of these documents match standard templates, lawyers must manually read every line to find the 20% containing dangerous deviations (e.g. unilateral indemnities, perpetual terms, or unfavorable jurisdictions).

The goal was to design an intelligent, interactive assistant that provides:

- Instant visual diffs against approved boilerplate documents.
- AI-generated summaries, risk scores, and priority ratings.
- Interactive suggested amends that can be merged directly into the document editor.

🤖 2. The AI Brainstorming Phase (Manus & Gemini)

To flesh out this concept, we engaged in an agentic workflow using AI assistants like **Manus** and **Google Gemini**:

- **Manus / Agentic Ingestion:** Used to research the initial structure of legal NDAs and Order Forms, helping draft the approved template boilerplates.
- **Gemini / Framework Design:** Brainstormed and drafted the risk classification schemas. This collaboration produced the strategic documents:
    - `Contract Priority Framework and Red Flag Taxonomy for AI-`

Assisted Review.md

- AI Action Plan for Streamlining Legal Contract Review.md

⚙️ 3. Prompt Engineering & API Parameters

A key decision was connecting the system to real LLMs (Gemini, Claude, ChatGPT, LLaMA). To get structured contract audits, we crafted a strict system instruction prompt. The AI models are called using the following parameters and guidelines:

LLM Prompt Parameters:

- **Model Choice:** Defaulting to `gemini-1.5-flash` for rapid, cost-efficient token parsing.

- **JSON Output Constraint:** Models are instructed to return a strictly structured JSON object (using `response_format: { type: "json_object" }` for OpenAI and prompt formatting for Gemini/Claude) containing:

```
{
  "summary": "Concise executive summary of risks...",
  "priority": "High" | "Medium" | "Low",
  "riskScore": 0-100,
  "redFlags": [
    { "category": "...", "severity": "...", "title": "...", "clause": "...", "description": "...", "suggestedAmend": "..." }
  ]
}
```

🔄 4. Iterative Corrections & Solving Pitfalls

Building the app involved resolving several technical challenges during development:

A. Resolving Browser CORS Policy Blocks

The Problem: Directly calling OpenAI or Anthropic APIs from client-side React code triggers Cross-Origin Resource Sharing (CORS) blocks in modern browsers.

The Correction: Implemented a fail-safe exception handler. If a network fetch fails due to CORS, the terminal console logs the error, and a high-fidelity local heuristics parser automatically runs. This preserves the UI state and generates accurate simulation flags without interrupting the user flow.

B. Native Dropdown White-on-White Bug

The Problem: In dark-mode glassmorphic themes, native browser select dropdown options frequently render white text on a white background, making options invisible.

The Correction: Added explicit CSS declarations targeting `.form-select option`, forcing a slate background (`#0f172a`) and white text (`#ffffff`) across all browsers.

C. Local State & Cache Recovery

The Problem: During hot reloads, old browser state fragments occasionally cause React render loops or null reference errors.

The Correction: Implemented a global React `ErrorBoundary` component. If the application crashes, a fallback UI displays the stack trace along with a **"Clear Cache & Hard Reload"** button that clears storage and forces a complete refresh.